

Senior Full Stack Developer

Technical Assignment

FeedbackHub

1. Introduction

The application you are asked to build is deliberately straightforward. It is a well-understood kind of web application, and we expect that modern AI coding tools can produce a working version of it quickly. That is intentional. We are not testing whether you can solve a hard algorithmic problem. We are assessing the judgment, discipline and craft you apply on top of what any competent developer — or any competent AI assistant — can already produce.

Concretely, we care about:

- The architectural and technical decisions you make, and your reasoning for them
- The quality and consistency of what you deliver
- **How you work with AI tools** — where you delegate, where you don't, how you verify, and how you stay in control of your own codebase
- How you handle scope and ambiguity

We expect and encourage you to use AI assistance. Using it well is part of the role. Section 6 describes what we need you to document about that process, and it carries real weight in our evaluation.

3. The Product

FeedbackHub is an internal product feedback board.

Employees submit feature requests and product feedback. Everyone can browse what has already been submitted, upvote requests they care about, and discuss them in comments. A product team member — an **admin** — triages incoming requests: setting their status, curating categories, and moderating content.

The purpose is to stop the same suggestion arriving five times by email and to make it visible what is actually being worked on.

3.1 Core Entity — Feedback Request

Field	Notes
Title	Short summary
Description	Longer free text
Category	e.g. Bug, Feature, Improvement, Question — configurable by admins

Field	Notes
Status	e.g. New, Under Review, Planned, In Progress, Done, Declined — configurable by admins
Author	The user who submitted it
Vote count	Derived from individual user votes
Comment count	Derived
Pinned	Admin-controlled
Timestamps	Created / last updated

3.2 Supporting Entities

- **Vote** — a user's upvote on a request. A user may vote for a given request at most once, and may withdraw their vote.
- **Comment** — free text on a request, by an author, with timestamps. Editable and deletable by its author.

3.3 Roles

- **User** — submits, browses, votes, comments; manages their own content and their own profile and settings.
- **Admin** — everything a user can do, plus triage and moderation, plus application configuration.

3.4 Core User Journeys

User

1. Registers or signs in through the identity provider.
2. Lands on the request list: sorted, filterable by status and category, searchable by text, paginated.
3. Opens a request, reads the discussion, upvotes it, adds a comment.
4. Submits a new request; sees it appear in the list.
5. Finds their own earlier request, edits its description, and later deletes it.
6. Updates their profile and application preferences.

Admin

1. Signs in and reviews recently submitted requests.
2. Changes a request's status and pins an important one to the top of the list.
3. Deletes an inappropriate comment.
4. Adds a new category and retires an unused one.
5. Adjusts application-wide settings.

3.5 Settings

User-level: display name, avatar or initials, theme (light/dark/system), language, default list sorting and filters, email notification preferences (e.g. on comments to my requests), account deletion.

Admin-level: categories and statuses (the taxonomy users select from), registration policy (open / invite-only / restricted to specific email domains), whether comments require approval, rate limits on submissions per user, and at least one feature flag that visibly changes application behavior when toggled.

We are interested in *where* configuration lives, how it is resolved between global defaults and user overrides, and how the frontend obtains it without a chain of blocking requests on startup.

4. Technical Requirements

4.1 Stack

- **Frontend:** Angular. Justify version used. Your choice of state management, styling and component approach.
- **Backend:** Node.js preferred. If you choose otherwise, justify the choice in writing.
- **Authentication:** an open-source identity, Support email/password sign-in and at least one social provider. **Do not implement authentication primitives yourself.**
- **Database:** your choice; justify it.
- **Deployment:** cloud-native. Containerized, environment-driven configuration, orchestrator-ready. A local Kubernetes distribution (kind, k3d, minikube) is entirely acceptable — we are assessing your manifests and configuration approach.

4.2 Architecture

A service-oriented decomposition is preferred, but we want your *reasoning* more than a particular shape. Propose a decomposition, define the boundaries, and defend them. If you believe fewer services — or a modular monolith with clearly identified seams — is the correct engineering decision for an application of this size, **make that argument**. A well-reasoned case for simplicity will score better than an unjustified split into multiple services.

4.3 Running the Application

Documented commands should bring the whole system up locally, with seed data.

Clearly defined requirements.

5. Deliverables

1. **Source code**, in a single **Git repository with real commit history**. Please do not squash your work into one commit.
2. **README.md** — what it is, how to run it, how to run the tests, what works, what doesn't.
3. **DECISIONS.md** — sample of decisions that mattered, context, options considered, decision, consequences. This should include your architectural choices and your reasoning about it.
4. **SCOPE.md** — what you built, what you deliberately did not build, what you would do next with another week, and any assumptions you made. If anything in this brief was ambiguous, this is where you tell us how you interpreted it.
5. **AI_COLLABORATION.md** — see Section 6.
6. **API specification (Optional)** OpenAPI or equivalent.
7. **Deployment artifacts** — Dockerfiles, compose file, Kubernetes manifests.

6. AI Collaboration Write-Up

This is a required deliverable and a significant part of the evaluation.

We use AI tools daily and we expect you to. What we want to understand is how you direct them, where you draw lines, and how you verify what comes back. Please write `AI_COLLABORATION.md` covering:

6.1 Tooling and division of labor

Which tools you used, for what kinds of work, and what you choose to do yourself if needed and why.

6.2 Your working method

How you operated: did you plan first and generate against the plan, generate and refactor, work file-by-file, or something else? Did your method change during the assignment? What context did you give the model?

6.3 Three worked examples

For three non-trivial parts of the application, show the prompt or prompts you used (verbatim or lightly trimmed), what came back, and what you did with it. We are interested in the gap between the first output and what you shipped.

6.4 Failures, if you hit any

If AI output was wrong at any point — a hallucinated API, a subtly broken check, a plausible but incorrect configuration, an outdated pattern — tell us about a couple of the more interesting cases: how you noticed, what it cost you, and whether it changed how you worked afterwards. If nothing of the kind came up, a sentence saying so is fine.

6.5 A rejection, if one occurred

If the AI produced something that worked and you replaced it anyway, tell us what was wrong with it and what you did instead. Skip this if it didn't happen.

6.7 Attribution in history

Please mark AI-heavy commits in some consistent way — a message prefix, a trailer, whatever you prefer — and describe your convention in the README.

Honesty is the point here. “I generated the entire CRUD layer with three prompts and reviewed it in twenty minutes” is a perfectly good answer if it is true and if you can defend the result.

7. How We Evaluate

AI collaboration carries the most weight. We are looking for depth and honesty in your write-up, and for evidence that you were directing your tools rather than accepting whatever they produced.

Engineering quality matters roughly as much. Consistency across the codebase, error handling, naming, readability, and a commit history that tells a story.

Correctness and security come next. Authorization enforced on the server rather than hidden in the UI, sensible validation, no trivially exploitable endpoints.

User experience is part of the deliverable, not polish to add later. Loading, empty and error states, keyboard accessibility, responsive layout, validation messages that a user can act on.

Architecture and decisions are judged on the quality of your reasoning in `DECISIONS.md` and whether your decomposition suits a problem of this size — not on how many services you created.

Scope and communication — the clarity of `SCOPE.md`, how you handled anything ambiguous, and whether you were honest about what is unfinished.

Two things worth stating plainly:

8. Submission

Send us a link to a Git repository — GitHub, GitLab or a bundle, your choice. If it is private, we will provide accounts to add.

If anything in this brief is unclear, **ask us**. Sending questions is not a mark against you.

Good luck — we look forward to seeing how you work.